

TechInterview Knowledge Base

Интеллектуальная база знаний для подготовки к техническим собеседованиям

Команда

3 инженера

Язык

Python

Интерфейс

CLI-инструмент

Тесты

177 passed

Проблема, Цели и Архитектура

Проблема

- | | |
|--|---|
| → Подготовка к собеседованиям разрознена — заметки, ссылки, случайные материалы | → Нет единого стандарта материалов и форматов |
| → Кандидаты тратят время на поиск, повторение и ручную структуризацию | → Решение: единая БЗ с валидацией, поиском и шаблонами |

Цели проекта

- | | |
|---|--|
| 01 | 02 |
| Единый формат | Быстрый поиск |
| Markdown + YAML frontmatter для каждой единицы знаний | Релевантный отбор материалов по роли, уровню, домену |
| 03 | 04 |
| Контроль качества | Генерация и реестр |
| Валидация метаданных, ссылок и структуры | Создание новых единиц, управление ID и аналитика |

Предметные области по ТЗ

CS Fundamentals

Фундаментальные основы Computer Science

Architecture / System Design

Проектирование и архитектура программных систем

Interview Process

Процесс собеседования и soft skills

Глобальный уровень

Терминология, связи, персонализированные траектории

Структура проекта и модель данных

Архитектура модулей

 `main.py`

Точка входа CLI. Argparse, маршрутизация команд, загрузка данных.

 Пакет `kb/`

`parser.py · validator.py · search.py · registry.py · generator.py`

 `tests/`

Автоматические тесты всех модулей. **177 passed** в 0.08s.

 Модули разделены по ответственности: вход, логика знаний и контроль качества.

Модель данных: единица знаний

Формат: **Markdown** + **YAML frontmatter**

Идентификатор и метаданные

`id, title, domain, category, difficulty`

Связи и контекст

`tags, roles, related, prerequisites`

Источники и статус

`sources → ссылочная база, status → жизненный цикл материала`

```
---
id: CS-CONCEPT-007
title: "Deadlock: причины и решения"
domain: cs
category: concept
difficulty: middle
tags: [deadlock, concurrency]
roles: [backend, sre]
interview_stage: technical
frequency: high
related: [CS-CONCEPT-005]
prerequisites: []
sources:
  - "OSTEP, Chapter 32"
status: review
---
```

CLI-команды

 **search**
Подобрать материалы под роль, уровень, этап

 **validate**
Найти ошибки и предупреждения в метаданных

 **registry**
Централизованный реестр ID

 **new**
Создать новую единицу по шаблону

 **stats**
Обзор состояния базы знаний

Технологический стек и инструменты разработки

Наша платформа использует компактный и эффективный набор технологий, чтобы обеспечить высокую производительность, структурированное управление контентом и удобную среду для разработчиков.



Основной язык

Python

Основа всей системы, которая обеспечивает серверную логику, обработку данных и командные интерфейсы с максимальной гибкостью и удобством поддержки.



CLI-фреймворк

argparse

Применяется для создания надежного и удобного командного интерфейса, обеспечивая эффективное взаимодействие со всеми возможностями платформы.



Формат контента

Markdown & YAML

Использование Markdown для текстового контента и YAML для структурированных метаданных frontmatter, что делает базу знаний удобной для чтения человеком и обработки машиной.



Тестирование и QA

pytest

Используется для комплексного модульного и интеграционного тестирования, обеспечивая качество кода, надежность и предотвращение регрессий по всей кодовой базе.

Тестирование и текущее состояние

Покрывтие тестами



Парсинг

Тесты `parser.py`: корректная загрузка YAML, обработка ошибок, `skip_errors`



Поиск

Тесты `search.py`: фильтры, `scoring`, сортировка результатов



Валидация

Тесты `validator.py`: правила ID, допустимые значения, ссылочная целостность



Реестр и генерация

Тесты `registry.py` и `generator.py`: ID-коллизии, шаблоны

Главный маркер качества

177

passed tests · 0.08s



✓ Все тесты проходят. Ни одного сбоя.

Текущее состояние базы знаний

60

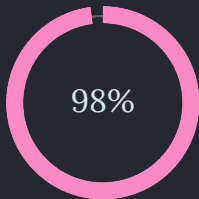
единиц знаний
Корректно парсируемых файлов

20

CS
CS Fundamentals



Тесты проходят успешно



Покрывтие ключевых модулей

20

arch
Архитектура / System Design

20

process
Interview Process



Дубли ID в реестре

approved: 50

Полностью подтверждённых единиц

draft: 5

В черновике

review: 5

На проверке — зона роста контента

Дубли ID: 0

Нет конфликтов в реестре



⚠ Часть файлов пока без полного YAML frontmatter — это запланированная зона развития контентной базы.

Проект в действии

Добавить новую единицу знаний

`new` → заполнение шаблона → `validate` → `registry`

Автоматический ID, корректный frontmatter, quality gate перед добавлением в базу.

Найти материал под интервью

`search --domain arch --category sysdesign`
`--difficulty middle --verbose`

Scoring, фильтры и объяснимый результат для быстрого поиска нужного материала.

Проверить «здоровье» базы знаний

`stats + validate`

Мгновенный обзор 61 единицы: распределение по доменам, статусам, ошибки и предупреждения.

❗ **Быстрый старт:** любому участнику команды достаточно одной команды, чтобы выполнить нужный сценарий.

✅ **Стабильность процесса:** CLI-интерфейс делает операции повторяемыми, понятными и стандартизированными.

Зона ответственности

search.py

registry.py



Поиск

Быстрый доступ к материалам по базе знаний



Реестр

Управляемая система идентификаторов

search.py — гибкая навигация

- Реализован SearchQuery с фильтрами: domain, category, difficulty, role, tags, text, status, frequency, interview_stage
- Система **scoring**: точные совпадения дают высокий вес, строгие фильтры исключают нерелевантные результаты
- Результаты **сортируются по убыванию** релевантности
- Обёртки: search_by_domain, search_by_tag, search_by_role, search_by_text
- Verbose-вывод: пользователь видит score и совпавшие параметры

registry.py — консистентность базы

- Построение реестра из всех единиц знаний, **проверка дублей ID**
- Поиск записи по ID, **генерация следующего корректного ID** по домену и категории
- Статистика по доменам, категориям и статусам; **экспорт Markdown-таблицы**

✔ **Эффект вклада:** search.py ускоряет навигацию и персонализирует отбор. registry.py устраняет коллизии ID и обеспечивает прозрачный аудит. Слой «доступ + учёт» стал устойчивым и масштабируемым.



Парсинг и валидация

Слой, который превращает исходные материалы в предсказуемые, проверяемые и готовые к поиску знания.

Зона ответственности

parser.py validator.py



Парсинг

Надёжная загрузка данных,
ingestion-пайплайн



Валидация

Контроль качества, правила и
quality gate



`parser.py` — ingestion-слой

Markdown с YAML frontmatter → единая модель KnowledgeUnit



`validator.py` — quality

gate
Проверка структуры, связей и полноты контента

`parse_file`, `parse_directory`, `parse_file_safe`

`skip_errors` для массовой загрузки без
остановки пайплайна

Валидация `related` и `prerequisites` —
целостность сети знаний

`validate_all`: дубли ID, **errors vs warnings**,
структурированный отчёт



Эффект вклада: данные стали предсказуемыми, контент — машинно-обрабатываемым, ошибки выявляются на раннем этапе. Слой «данные + качество» стал системным фундаментом платформы.

Зона ответственности

generator.py
main.py

2

Core modules

generator.py + main.py

5

CLI commands

search, validate, registry, new, stats



Генерация

Автоматизация создания
контента по шаблонам

CLI

Единая точка входа,
production-ready workflow

generator.py — автоматизация контента

Генерация новых единиц знаний по
шаблонам с автоматическим **frontmatter**Мэппинг доменов и категорий в директории,
транслитерация, **slug-генерация**Интеграция с **registry.py** для присвоения
следующего корректного IDРежим **--dry-run**: файл не создаётся,
превью результата

Template

Берём шаблон и заполняем структуру



Preview

Проверяем результат в dry-run



ID

Получаем следующий корректный идентификатор

Эффект вклада: создание материалов ускорено шаблонами, все модули объединены в единый интерфейс. Инженер 3 замкнул full-stack CLI-цикл платформы.

Командный итог и Roadmap

Инженер 1

`search.py + registry.py`

Доступ и индексируемость



Инженер 2

`parser.py + validator.py`

Структурность и quality gate



Инженер 3

`generator.py + main.py`

Автоматизация и CLI-интеграция

Roadmap

01

Полный frontmatter

Довести все Markdown-файлы до полного
YAML-стандарта (сейчас 51 в review)

03

Метрики качества

Дополнительные аналитические метрики контента в `stats`

02

Расширение контента

Увеличить объём единиц знаний по ТЗ в
доменах `arch`, `cs`, `process`

Итог

177

passed tests

Инженерная зрелость

60

единиц знаний

И база растёт



🚀 Проект готов как фундамент масштабируемой базы подготовки к собеседованиям.

Будущее автоматизации: Расширение платформы

План развития для дальнейшего повышения эффективности и персонализации в системе управления знаниями.

1



Автоматизированное пополнение базы

Автоматический сбор, парсинг и структурирование информации из различных источников (документация, статьи, репозитории) для непрерывного обновления знаний.

2



Интеллектуальные рекомендации с RAG

Интеграция Retrieval-Augmented Generation для выдачи контекстно-зависимых и точных рекомендаций, улучшающих релевантность поиска и генерацию ответов.

3



Персонализированные траектории обучения

Динамическое формирование учебных планов, адаптированных под индивидуальные навыки, опыт и цели каждого инженера для максимально эффективного развития.

4



Адаптация к стилям обучения

Система будет анализировать предпочтения пользователя (визуальный, аудиальный, практический) и темп усвоения материала, предлагая наиболее эффективные форматы и скорость обучения.



Эти направления развития обеспечат создание **саморазвивающейся и глубоко персонализированной** платформы для подготовки инженеров, значительно повышая их эффективность.